Future Generation Computer Systems 170 (2025) 107833



Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs





Enhancing the output of time series forecasting algorithms for cloud resource provisioning

Ferran Agulló [a,b,*], Alberto Gutierrez-Torre [a], Jordi Torres [a,b], Josep Ll. Berral [a,b]

[a] Barcelona Supercomputing Center, Barcelona, Spain
[b] Universitat Politècnica de Catalunya - BarcelonaTech (UPC), Barcelona, Spain

ARTICLE INFO

Keywords:
Time series forecasting
Resource forecasting
Cloud provisioning
Resource autoscaling
Deep learning
Explainability

ABSTRACT

Forecasting the resource consumption of workloads is a frequent approach in the cloud provisioning field. Ideally, such predictions allow obtaining a more accurate scheduling and management of resources in a computing cluster. However, the current approaches fail to properly forecast the future consumption in areas where sudden increases of consumption are present, *i.e.*, spikes. Even, commonly employed metrics lack the ability to properly evaluate sharp behaviours in the traces. This may generate resource starvation problems in the running workloads and decreases the Quality of Service (QoS) provided to external users. To address this issue, we propose two strategies that modify the outputs of forecasting algorithms without changing the algorithms' internals. The new outputs considerably enhance the prediction of sudden increases, duplicating the F1 score metric in average for all tested algorithms. This improvement in the handling of spikes comes with an increased over-provision of resources. Nevertheless, the proposed strategies give the user an easy way to control this trade-off between predicting spikes and the amount of over-provision. The user can decide which is the right balance that better fits the requirements of its specific scenario. Furthermore, we propose a new evaluation methodology that better assesses the behaviour of forecasting algorithms in cloud traces, especially focused on the performance around increases of consumption, and we give insights on the reasons behind the predictions of the algorithms with the application of explainability techniques. The code repository of this work can be accessed through GitHub at this link https://github.com/FerranAgulloLopez/ResourceForecasting.

1. Introduction

Since the last decade, cloud computing has emerged as the preferred option for companies to handle and deploy their software stack. It provides a new on-demand approach for the utilization of resources that distances the developers from the real low-level infrastructure [1]. In addition, cloud computing accelerates the deployment phase of software and, moreover, commonly reduces the cost that entails maintaining a local corporate cluster. Good examples of companies that provide these services are Google, Alibaba, Amazon and Microsoft, usually referred to as cloud providers. These corporations own enormous data-centres with hundreds of thousands of computing nodes that have a huge demand in resources and, thus, a rather large energy consumption. This consumption is most probable to increase in the upcoming years alongside their growth in the market [2,3]. For that reason, the research in the cloud computing field has deviated to increase the efficiency of these centres, trying to reduce the overall energy consumption and trying to use the available hardware in the most cost-effective manner. These efforts can entail a reduction in the cost of maintaining a computing infrastructure and a minimization of the environmental footprint of cloud providers.

One of the principal efficiency challenges is the proper allocation of resources to the workloads that run in the computing nodes. Every workload has a different purpose with its own specific characteristics. They undergo multiple phases of execution that require distinct resource demands [4]. Current scientific applications, for example, can contain hundreds of thousands of interdependent tasks, which pose a significant challenge in defining a shared resource allocation for the full length of their execution [5]. If this allocation is below the highest point of resource consumption, the workload will suffer from resource throttling or starvation (under-provisioning), which may impact the Quality of Service (QoS) of the end users. However, on the other hand, if the allocation is over this point, the majority of resources will not be employed during the other phases of the execution and the cluster resources will be underutilized (over-provisioning). In order

* Corresponding author.
E-mail addresses: ferran.agullo@bsc.es (F. Agulló), alberto.gutierrez@bsc.es (A. Gutierrez-Torre), jordi.torres@bsc.es, jordi.torres@upc.edu (J. Torres), josep.berral@bsc.es, josep.ll.berral@upc.edu (J.L. Berral).

https://doi.org/10.1016/j.future.2025.107833
Received 30 July 2024; Received in revised form 4 February 2025; Accepted 22 March 2025
Available online 2 April 2025
0167-739X/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

to overcome this dynamic resource usage, the standard approach in the literature consists on employing forecasting algorithms to predict the resource consumption of the running workloads. Cloud providers can leverage this future information to accommodate the right amount of resources to fluctuating workloads at each moment of time, decrease the amount of over-provisioning and reduce starvation problems due to under-provision of resources. In addition, these forecasts can be leveraged to decide the best placement of the running workloads across the available nodes, and dynamically auto-scale and down-scale applications according to the current usage (elasticity) [6].

1.1. Motivation

The majority of these resource forecasting algorithms focus on obtaining forecasts that are in average close to the real usage. They do not concentrate their efforts in predicting sudden increases of consumption (spikes of usage). They often forecast these spikes after they have taken place or with the wrong height. The left cases of Fig. 1 show an illustration of these recurrent behaviours, which takes as example the charts exposed in the works from Khan et al. [7], Janardhanan et al. [8], Ouhame et al. [9] and Saxena et al. [10]. This inability to correctly anticipate spikes of consumption leaves no time for the system to take any corrective actions necessary to ensure that the workload can receive the needed resources and handle the increased load. The node hosting the workload may lack sufficient resources, or existing resources might be occupied by other concurrently running workloads. Notice that cloud providers usually reutilize unused resources on other workloads. In the case of the Google [11,12] and Alibaba [13,14] clusters, low-priority workloads exploit the free resources requested by high-priority workloads.[1] That is to say, the sudden increase of a high-priority workload may be inside the paid resource limits, but these resources may be allocated to other workloads at the moment of the increase. Without the proper anticipation, the cloud provider cannot reassign back the needed resources and the high priority workload will suffer from resource throttling or starvation, infringing the Service Level Objective/Agreement (SLO/SLA) between the cloud provider and its users, and reducing the QoS of external users. This behaviour, between others, can result in a permanent loss of a portion of the revenue of cloud providers [15].

In the same line, the most used evaluation metrics in the literature, Mean Squared Error (MSE) and Cross Entropy (CE), or some of their variants like Mean Absolute Percentage Error (MAPE) and Root Mean Squared Error (RMSE) [6,16], lack the ability to properly evaluate sharp behaviours in the traces, as stated in the work from Vincent Le Guen et al. [17]. They give more importance to the average accurateness instead of accurately evaluating sudden changes. Section 4.2 shows empirical results of why average metrics, like the Mean Squared Error (MSE), are not appropriate for properly evaluating sudden increases of consumption in cloud traces. They can give higher scores to forecasts that are not performing correctly in these areas.

1.2. Contributions

In this work, we focus on generating forecasts of resource consumption that correctly predict sudden increases of consumption. We propose two strategies that modify the output of forecasting algorithms towards the right case of Fig. 1. The new forecasts are able to anticipate a bigger number of spikes beforehand and with the right height. In addition, these forecasts tend to always provide values over the real usage by a determined factor. We consider that in the cloud provisioning field, under-provisioning can have a greater impact in production systems than over-provisioning. Nevertheless, this factor of over-provisioning and the amount of anticipation before the spikes can be configured in the two proposed strategies depending on the final use case scenario and its specific requirements. The system administrator can decide the desired trade-off between the two aspects. The modified forecasts can help to enhance the overall management and efficiency of the computing nodes in a cloud environment, putting special attention in reducing starvation problems and SLA violations. Although the proposed strategies can work with all types of resources, we opted for testing them with CPU above others due to its flexibility when dynamically sharing resources between co-allocated workloads. The two strategies modify the training goal during the training phase of forecasting algorithms towards the desired output. They can be applied to a wide range of models without modifying their internal structures. The first strategy is generic and can be used with any model, while the second is specifically designed for deep learning models.

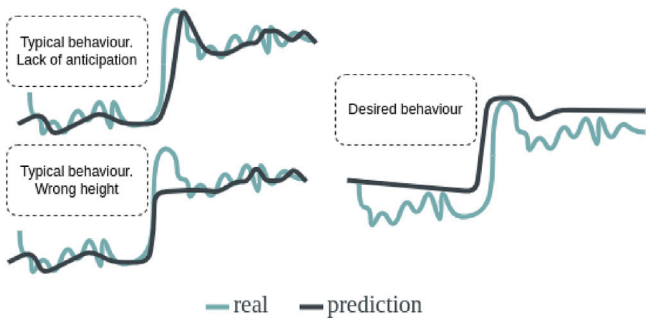


Fig. 1. Illustration that shows three different behaviours around a spike of consumption. The two left examples denote the typical focus on the literature, obtaining a forecast that is close to the real usage in average. The right example focuses in foreseeing increases of consumption with anticipation and with the right height, the desired behaviour in this work.

In the same direction, we propose a new evaluation strategy that aims to correctly evaluate, not only a close allocation in average, but also the proper prediction of the sudden increases of consumption. This evaluation gives a more deep insight into the trade-off between over/under provisioning and the proper forecasting of spikes. With this approach, the final user can decide what forecasting algorithm is more closed to its specific scenario requirements.

Lastly, this work considers the state-of-the-art models from the general domain of time series forecasting, making use of the new multi-layered convolutional models and the Transformer architecture. In addition, explainability techniques are used to get a better view on the latent reasons behind the predictions of the models.

In summary, this work provides the following key contributions:

- Two strategies that modify the output of time series forecasting algorithms to better predict sudden increases of consumption and to avoid situations of under-provisioning in cloud traces. They can be applied to a wide range of models without modifying their internal structures. Furthermore, they can be configured to obtain the desired trade-off between forecasting spikes and reducing the amount of over-provisioning depending on the requirements of the specific use case.
- A new evaluation strategy that can assess the proper prediction of sudden increases of consumption and an overall good forecasting of resources in cloud traces. It gives detailed and individual information regarding the different aspects that determine a good forecast in the cloud provisioning field.
- The employment of explainability algorithms to better understand the reasons behind the predictions of deep learning models in cloud traces.

[1] For the most part, low-priority workloads consist of batch jobs or computing processes with high tolerance to resource availability changes, which have a much lower QoS to fulfil. On the other hand, high priority workloads are processes that are accessed by external users, like websites or public APIs, that must provide a good QoS

1.3. Related work

There are plenty of contributions in the literature that propose methods to forecast the resource consumption of workloads, taking into consideration the different surveys of the topic [4,6,16,18–20]. They tackle the challenge as a time series forecasting problem, creating time series with the past observations of resource usage. The majority of methods are based on classical time series forecasting models [21–25], machine learning variants like Random Forests or Support Vector Machines [26–29] or ensembles/combinations of them [30–34]. A clear example is the Facebook Prophet adaptation to cloud resource management [35] or the work from the Microsoft team for improving the Azure scheduler [36]. The Facebook Prophet uses a modification of the well-known ARIMA model [37], whereas the Microsoft team applies the algorithms Random Forest and Extreme Gradient Boosting Tree.

As well, there is a substantial list of contributions that propose the usage of deep learning methods [8–10,38–42]. An interesting approach comes from the work of Chen et al. [43] that proposes an ensemble of a global and a local model. The global model contains the information from all the set of time series, whereas the local model only gives information about the learnt patterns from similar time series. It is also interesting to point out the comparison between multiple simple classic machine learning algorithms and deep learning models performed in the work of Khan et al. [7]. In their study, the GRU model came on top [44], an improved version of the well known LSTM recursive model from the domain of natural language processing (convolutional networks were not considered in the study).

It is important to mention the considerable gap in the literature with regard to the new state-of-the-art models that make up the general domain of time series forecasting. As discussed, there are plenty of contributions that propose the usage of deep learning methods, although, the multi-layered convolutional networks and complex Transformer [45] architectures have not been tested in a fair amount. Certainly, there are some works that propose these types of networks [46–48], but they are not yet accepted by the larger community in the field of cloud provisioning. Some good examples of these models are the SCINet [49] and the Informer [50], they keep some of the highest scores in the most popular benchmark of time series forecasting, the ETDataset. In this regard, it is also quite significant to acknowledge the contribution from Zeng et al. [51] that questions the validity of the complex Transformer models in time series forecasting tasks. It still needs to be validated by the research community, but they outperformed the Transformer architectures with embarrassingly simple one-layer fully connected networks.

1.3.1. Prediction of spikes of usage

As stated in the Motivation section, the majority of these forecasting algorithms focus on predicting close to the real usage in average, and do not properly anticipate sudden increases of consumption, which consists the principal goal of this work. Nevertheless, we can find some contributions that try to detect or predict spikes of usage, some of them employing the term "anomalies" to describe these increases. We can differentiate between reactive [52–55] and predictive [56,57] methods. Reactive methods identify workload spikes after they occur, analysing resource usage retrospectively. Predictive methods, in contrast, aim to forecast potential spikes before they happen, enabling anticipatory resource allocation actions. For example, in the work from Zhao et al. [56], they propose a threshold-based approach. Presuming that a spike is preceded by a period of anomalous resource usage, they forecast a spike if the observed resource usage deviates beyond a predefined threshold from the expected behaviour of the workload. This expected behaviour is extracted through a LSTM network trained in the trace data. Alternatively, in the work from Patil and Patil [57], they calculate the time required for current consumption levels to reach some predefined thresholds assuming the ongoing usage increase rate remains constant. A spike is predicted if a relatively short time is estimated, also considering the system's migration time as an additional factor.

While these existing studies offer valuable insights, they fall short in several key aspects. Notably, either they do not predict the height of the spikes, reducing the problem to binary classification, or they are not able to forecast seasonal or other non-stationary patterns. To the contrary, our proposed strategies try to predict the spikes in magnitude before, during and after they occur. In addition, they are able to leverage the analytical power of the chosen forecasting algorithm to recognize seasonalities or other complex patterns containing non-linear relationships.

1.3.2. Evaluating predictions of spikes of usage

In regard to the evaluation metrics, as aforementioned, the majority of works use average metrics, like the MSE, that do not properly evaluate spikes of resource usage. It is also very common to employ metrics specific to the cloud field. Usually, different forms of quantifying the over/under provisioning of the workloads [22,31]. They differentiate between predicting over and under the real usage, which has different implications in a cloud scenario, but they still are average metrics that do not correctly evaluate sharp behaviours. In the same direction, other works calculate the SLO/SLA violations, as in [23,24]. These violations count the number of times the predictions are located under the real usage, quite analogous to the under-provisioning concept. In a similar fashion, some works quantify the success in the cost of money, taking as an example a specific cloud environment [23,24].

Alternatively, other contributions propose scheduling experiments to test the final performance in the user side. The works from Calheiros et al. [58] and Zhang et al. [59] are representative examples. The first work used the number of rejected requests as the evaluation metric, whereas in the second work the latency was employed instead. Indeed, this last work, proposed the usage of two distinct models. A first model that predicted possible violations as a binary problem, and a second model that forecasted the actual future resource consumption. Interestingly, the first model, built with Boosted Trees, used the latent space found by the forecasting predictor build of convolutional layers. Nevertheless, it is not clear whether the forecasting predictor was actually helpful for the final results.

While these evaluations assess various aspects of workload forecasting, they do not specifically address the need for accurate prediction of sudden increases. Our proposed evaluation checks if the initial point that marks a spike of usage is forecasted beforehand and with the right height. Forecasting it afterwards leaves no time for doing corrective actions and avoiding starvation in the workloads. Indeed, a dummy forecasting algorithm that uses the current usage as the future prediction will be able to predict all sudden increases just after they happen and with the correct height. To our knowledge, there is only one work that focuses on the same aspect as ours. That is the work from Mehta et al. [53]. They manually tagged the spikes of the dataset and employed the so-called precision and recall metrics to quantify their correct detection. In this way, the evaluation approach counted both the true positives and the false positives. A forecasting algorithm will only obtain a high score if it is able to anticipate the real spikes without over-predicting spikes that will never occur. They also quantify the distance in time and height of the correctly predicted spikes to the value of the real usage. Our evaluation approach is inspired by this work. The main difference is that our approach does not need to manually tag the sudden increases. It is able to extract them automatically from the real and forecasted series to calculate the misses and hits, which is indispensable for the datasets that are used in this work, that contain thousands of traces of more than 8 days long. In addition, we differentiate between predicting a spike over or under its real usage, because we consider that it can have distinct implications in a cloud provisioning scenario.

1.4. Outline

The remaining of this work is divided as follows: Section 2 describes the two output alteration strategies, Section 3 explains the new evaluation and Section 4 presents the performed experiments that validate the proposed contributions. Lastly, Section 5 and Section 6 finalize the article with the discussion and conclusions of the work.

2. Forecasting output alteration

This section describes the two proposed strategies that modify the output of forecasting algorithms: distinction between series and loss function modification. Both strategies allow the alteration of the training goal during the training phase of forecasting algorithms. In the experiments section, they are applied and tested in the cloud provisioning field, trying to obtain predictions similar to the right example of Fig. 1. Notice that they are two independent strategies, and they are applied and tested separately in this work.

2.1. Distinction between series

A model represents a transformation function in the form of $M(x) = y'$ where x and y' denote the input and output, respectively. In the Time Series Forecasting task, typically x refers to the past samples, whereas y' denotes the samples that are forecasted that should be equivalent to the future observations y, all coming from the original time series. Nevertheless, as introduced in Section 1.1, the final goal for cloud provisioning is to predict over the real usage by some considerable margin and with a proper anticipation to the different increases of consumption. Accurately predicting the original time series constitutes a different objective, usually even more challenging.

For that reason, for this first output alteration strategy, we make a distinction between the original series and the series that the model is meant to forecast, similarly to the Sequence-to-Sequence pattern. This new series, that we are naming as ***target series***, entails the outcome that is more adequate for the final use case at hand. It represents the desired target of the model, from where the future ground truth observations y will be taken. In addition, instead of using samples from the original series as the input of the model, a new series named as ***values series*** is employed alternatively. This new series should help the model better understand and recognize the patterns and sequences that are located in the past.

Both series are obtained by applying different transformations to the original time series. It is equivalent to a set of preprocessing steps, but the distinction between series allows focusing on different aspects. The transformations on the *target series* modify the objective of the model, whereas the transformations on the *values series* can simplify the model learning. Fig. 2 shows an example of the proposed distinction. In this case, the *values series* is created by applying a mean filter to the original series that only takes past observations, simplifying the harshness of the sequence, which may help the model better understand the general patterns of the time series. Simultaneously, the original series is modified using a maximum filter with both past and future samples, creating the *target series*. In this way, the model needs to learn to anticipate the increases in consumption and, also, to maintain them for a certain amount of time. Notice that, in order to apply this strategy in an online predictor, all applied transformations in the *target series* must be revertible to transform the output value to the original form, and the *values series* must never contain transformations that use future samples.

This differentiation enables the correction of the goal of the model and its adaptation to a specific real scenario. Allowing to focus the learning direction to the desired patterns and sequences, avoiding learning unnecessary elements or noise. This approach takes all the potential and strength from the model to learn the specific use case that may contain complex patterns with non-linear characteristics. In addition, when applying transformations to the *target series* that use information from future samples, the model can be able to anticipate situations that have not yet happened. In this way, the transformations to apply depend entirely on the final use case and the data that is being forecasted.

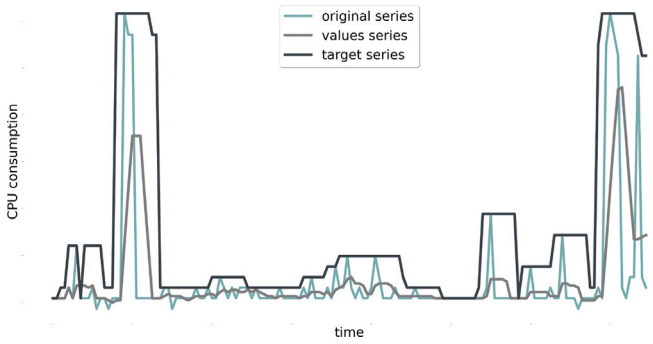


Fig. 2. Example of the first proposed output alteration strategy, where the three types of series can be seen: original, *values* and *target*. It is a sliced trace from the CPU consumption of a workload in the Alibaba dataset.

2.2. Loss function modification

As an alternative to the previous distinction between series, we also propose to alter the goal of the learning through the ground truth values that are applied in the loss function of deep learning models. For this strategy, we only focus on one-step forecasts and on the classification case, where the time series are discretized to simplify the model learning.

In one-step prediction scenarios, every observation y is forecasted individually by the model based on a set of previous observations x. The loss function evaluates the difference between the ground truth y and the output of the model y'. In the classification case, both y and y' are vectors of size n, where n is the total number of intervals resulting from the discretization step. Every value y_i $(0 \leq i < n)$ receives the empty value (0) except the interval that contains the future sample within its limits named as $i*$, which receives the full value (1). In this way, the model learns to only forecast the interval where the real sample is located. On the other hand, every value y'_i $(0 \leq i < n)$ represents the probability of being the correct interval that the model outputs for every interval i.

As aforementioned, in cloud provisioning, there is no such strict constraint to only predict the correct interval. Even more, we usually prefer to predict over the real usage than under it, in order to avoid resource starvation in the workloads. Therefore, we propose to portray the characteristics of the field directly in the ground truth vector y applied in the loss function of deep learning models. For instance, every value y_j $(i* \leq j < n)$ can be changed to hold the full value, changing the training goal to avoid situations of under-provisioning at all costs. Similarly, every value y_j over the correct interval can be modified to take a value in proportion to their proximity to $i*$. In this manner, the model learns to predict over the real usage, but the closest to its original value. We refer to this modified ground truth vector as $altereted_y$.

The final formulation of the loss function is depicted in Eq. (1). This formulation is equivalent to the one exposed in the work of Dastidar, et al. [60]. They employed it for transferring knowledge from a big network (teacher network) to a less sophisticated and simpler one (student network), a process typically known as Knowledge Distillation [61]. The loss function is modified so that the student network can learn directly from the latent space of the teacher network rather than from the ground truth. The loss function is composed of two different terms that are regulated by a constant α. In our case, the first term minimizes the distance between the output of the model $M(x)$ and the altered

ground truth *altereted_y*. The constant T is employed to control the temperature of the softmax, modifying the probability distribution over the classes [61]. On the other hand, the second term minimizes the output of the model $M(x)$ with the original ground truth y, which corresponds to the standard loss function employed in classification scenarios. The latter is useful to regulate the importance of the altered ground truth, creating a trade-off in the learning goal between the two terms.

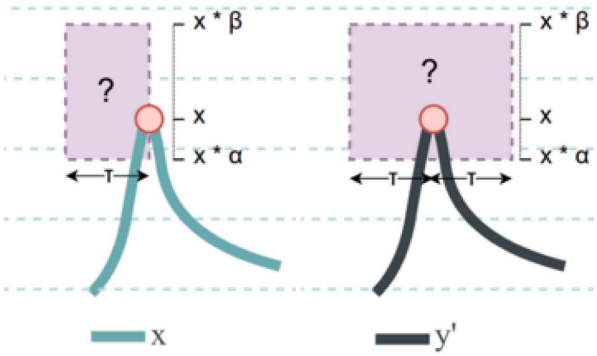
$$Loss\ function = \alpha * KLD\begin{pmatrix} M(x)/T, \\ altereted_y/T \end{pmatrix} \\ (1-\alpha) * CE(M(x), y) \tag{1}$$

3. New evaluation

As introduced in Sections 1.1 and 1.3.2, the majority of contributions in the literature use metrics such as Mean Squared Error (MSE), Cross Entropy (CE) and Accuracy, or some of their variants. However, these metrics lack the ability to properly evaluate sharp behaviour in the traces like spikes of consumption, as we see experimentally in Section 4.2. Therefore, we propose a new evaluation, focused on the cloud provisioning field, that can assess the proper prediction of the increases of consumption along an overall good forecasting of resources in the traces. The new evaluation is agnostic about the employed algorithm, it only requires the times series for the original and predicted consumption. For the equations and descriptions to come, the series corresponding to the original consumption is depicted as $[x_0, x_1, \dots, x_{n-2}, x_{n-1}]$ where n is the length of the trace. Besides, the predicted consumption is depicted in the form $[y'_0, y'_1, \dots, y'_{n-2}, y'_{n-1}]$. All values from both traces are assumed to be equal or over zero.

In order to assess the correct prediction of spikes, similar to the work from Mehta et al. [53], we use two distinct metrics, **spikes-precision** (spikes-prec) and **spikes-recall** (spikes-rec). These two metrics work as the two sides of the same coin, in accordance to the well-known precision and recall measures. The spikes-recall measures the fraction of spikes from the original series that are correctly predicted both in time and quantity of resources. On the other hand, to avoid scoring high an algorithm that always predicts spikes, the spikes-precision penalizes when the algorithm foresees increases that do not occur in the original trace. The steps to compute both metrics are presented in Algorithms 1 and 2. In summary, we use the standard deviation with the most recent group of observations as a reference to detect spikes, both in x and y'. Afterwards, we determine that a spike from the original series is correctly predicted if there is a value in y' with similar height and close in time (recall). Fig. 3.a shows the area where a similar value is searched. Notice that a spike in x is correctly predicted only if it is forecasted before it has occurred. On the other hand, Fig. 3.b shows the area that is searched to determine if a spike in y' is indeed related to the usage from x (precision). In this case, we also check future samples, we left the option for the model to predict beforehand. The width and height of the areas to search are defined by constants τ, α and β as shown in Algorithms 1 and 2, and can be tuned to the specific use case at hand.

Apart from assessing the proper forecasting of spikes of consumption, the proposed evaluation also checks whether the forecasts are close to the original values during the full length of the trace. Because predicting over and under the real usage has two opposite consequences, we use two new metrics to quantify this average margin of error: **over-provision** (over-prov) and **under-provision** (under-prov). They echo the direction of the literature and compute the margin of error on average when forecasting the resource consumption [22,31]. As shown in Eq. (4), the over-provision quantifies the area between the series x and y' when the former has an inferior value. Similarly, the under-provision measures the area between the two series when x has the superior value, presented in Eq. (5). Equivalent to the MSE metric, the outcome is the mean of the squares of the errors, penalizing



(a) Area to search to determine if a spike from x has been correctly predicted by y' (spikes-recall).

(b) Area to search to determine if a spike from y' has some similarity with the values in x (spikes-precision).

Fig. 3. Search areas used to compute the metrics spikes-precision and spikes-recall alongside the employed constants.

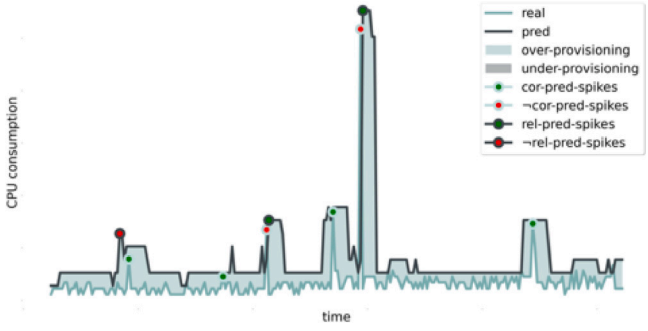


Fig. 4. Example of the behaviour of the proposed evaluation in a sliced trace from the Alibaba dataset. It shows the *real* (x) and *pred* (y') series alongside the produced over- and under-provisioning (the latter cannot be seen). In addition, the Figure shows the spikes of x that a correctly predicted (*cor-pred-spikes*), the spikes of x that are not correctly predicted (*¬cor-pred-spikes*), the spikes of y' that are close to the values in x (*rel-pred-spikes*) and the spikes of y' that are not close to the values in x (*¬rel-pred-spikes*).

to a greater extent the algorithms that fail by a greater margin. Aside from that, the difference between both traces is computed relatively to the first $((y'-x)/x)$. This formulation comes from the assumption that a computing node can support a larger spike if the workload has a greater consumption. Typically, the scheduler module gives a resource margin proportional to the consumption of the workload.

$$over\text{–}provision = \frac{1}{n}\sum_{t=0}^{n-1} max(\frac{y'[t]-x[t]}{x[t]}, 0)^2 \tag{4}$$

$$under\text{–}provision = \frac{1}{n}\sum_{t=0}^{n-1} min(\frac{y'[t]-x[t]}{x[t]}, 0)^2 \tag{5}$$

Fig. 4 shows an example of the proposed evaluation in a trace from the Alibaba dataset. The four metrics give enough and significant information about the characteristics of the forecasted resource consumption. It is possible to know if the algorithm tends to over-provision or under-provision and in what magnitude. As well, it is possible to be aware of the real performance in the detection of the increases of resource consumption, to better anticipate the required actions for a proper resource management in a computing cluster.

4. Experiments

This experiments section is divided into (1) general description of the settings used in the experiments (2) motivation for the new

Algorithm 1 Steps to compute the metric spikes-precision.

requires: x and y' series
constants: $\gamma, \sigma, \alpha, \beta, \tau$

step 1: compute *pred–spikes*
We perform a sequential pass through all observations of y', to determine the locations of the contained spikes. For each observation y'_k $(0 \leq k < n)$, we extract the lag window $[y'_k - \gamma, y'_k)$ $(lagW)$. In order to avoid the detection of contiguous / close spikes with similar heights, if a spike y'_j $(0 \leq j < n)$ has been detected between $y'_k - \gamma$ and y'_k, the lag window is reduced to $[y'_j, y'_k)$. Once extracted, we determine that y'_k is a spike if $y'_k - w_mean(lagW) > \sigma * w_std(lagW)$, where w_mean and w_std are the weighted mean and standard deviation of the lag window with weights equivalent to $[i = split, i < 1 + split, i + split]$, being $split$ equal to $1/len(lagW)$. Notice that only upward spikes are detected.

step 2: compute *rel–pred–spikes*
For each found spike in y', we compute whether it has some relation with the trace x. A detected spike y'_k is related if $\exists((y'_k - y'_k * \alpha) < x_g < (y'_k + y'_k * \beta))$ where $0 \leq (k - \tau) \leq g \leq (k + \tau) < n$ (search area shown in Figure 3.b).

step 3: compute *spikes–precision*
The metric is computed in the following way:

$$spikes\text{–}precision = \frac{rel\text{–}pred\text{–}spikes}{pred\text{–}spikes} \tag{2}$$

where *pred–spikes* is the number of found spikes in y', whereas *rel–pred–spikes* is the number of found spikes in y' that additionally have some relation with the trace x, that is to say, the predicted spikes that are both close in time and quantity to the real consumption.

Algorithm 2 Steps to compute the metric spikes-recall.

requires: x and y' series
constants: $\gamma, \sigma, \alpha, \beta, \tau$

step 1: compute *real–spikes*
The algorithm is equivalent to step 1 of Algorithm 1 using the x trace instead.

step 2: compute *cor–pred–spikes*
For each found spike in x, we compute whether it is correctly predicted by the trace y'. A real spike x_k is correctly predicted if $\exists((x_k - x_k * \alpha) < y'_g < (x_k + x_k * \beta))$ where $0 \leq (k - \tau) \leq g \leq k < n$ (search area shown in Figure 3.a). Notice that the main difference with the spikes-precision metric is that g cannot be further in the future than the found real spike.

step 3: compute *spikes–recall*
The metric is computed in the following way:

$$spikes\text{–}recall = \frac{cor\text{–}pred\text{–}spikes}{real\text{–}spikes} \tag{3}$$

where *real–spikes* is the number of found spikes in x, whereas *cor–pred–spikes* is the number of found spikes in x that additionally are correctly predicted by the trace y', that is to say, the real spikes that are predicted in advance and with similar quantity.

evaluation (3) analysis of the two proposed forecasting output alteration strategies and, (4) exploration of the models' behaviours with explainability techniques.

4.1. Experimental settings

4.1.1. Scope

The scope is reduced to a specific scenario in order to restrict the available possibilities and the magnitude of the required experiments. The chosen scope consists of forecasting the CPU consumption of the next sample for all the observations of every time series (one-step-ahead prediction). The methods take into account only past information from the corresponding series and global learnt patterns from all series of the training set.

4.1.2. Datasets

We conduct experiments on two widely used datasets that contain traces of resources that come directly from workloads: Google 2019 [62] and Alibaba 2018 [63]. Table 1 shows the general information of the two datasets once they have been processed. In both datasets, the feature to predict corresponds to the maximum usage of CPU in each moment of time. The preprocessing of both datasets was done taking into account the specific requirements of machine learning models, especially deep learning models, which diverge from the preprocessing requirements of classical methods like ARIMA, mainly by not requiring stationarity transformations.

The Google dataset from 2019 collects resource traces from workloads in eight different clusters for the full length of a month. We perform a subset of the full extent, taking into account only the first cluster (named as a) and selecting the time series corresponding to workloads with production tier instances that take longer than 15 days to complete.[2] For avoiding a possible bias, we randomly extract only one time series from the set of instances of every workload. Subsequently, because of notorious differences between the resulting time series, we select a cluster of time series with common patterns through a Principal Component Analysis (PCA), using the features of the Python library TSFEL [64]. Lastly, we perform feature reduction based on correlation, null imputation, time granularity regularization and at last, a standardization of the final values.

On the other hand, the Alibaba dataset from 2018 contains workload traces from around 4,000 machines for a period of 8 days long. Similarly to Google, we select the long-running workload instances that do not complete during the total period of 8 days, and we randomly choose one time series for every workload. In this way, we obtain a subset of long-running time series, whose predictions are more useful in a cloud scenario, with common patterns and a similar seasonality. Unlike in the Google dataset, we ignore the workload instances that have a maximum CPU usage very close to their average usage, which are traces with little interest in a dynamic allocation scenario. In addition, we filter out the time series that contain many null values and perform null imputation in the remaining. We reduced time distancing to windows of 5 min to make them equal to the Google dataset and, lastly, we standardize the resulting values.

[2] The selection is performed through the Google cloud query platform called BigQuery (https://cloud.google.com/bigquery) that allows to query the multiple dataset tables with SQL statements

Table 1
General information of the used datasets after preprocessing.

	Google	Alibaba
Number of traces	1,722	5,168
Number of features	14	18
Length	15-30 days	8 days
Window time	5 min	5 min
Disk size	1.8 GB	2 GB

The time series of both datasets are divided randomly into training, validation and testing sets with percentages 70%, 15% and 15% respectively. All results and figures that appear in this section come from the testing set. In order to work with the methods, we create windows for each observation of every time series, containing a lag size of past observations. For avoiding any bias, we extract the same number of windows for each time series.

To conclude, many of the experiments in this section discretize the output of the model to treat the challenge as a classification problem. This discretization is performed with 40 intervals of equal size that cover the range between the maximum value and the minimum value of the corresponding dataset.

4.1.3. Methods

While classical methods such as ARIMA have been widely used, we have opted for testing our contributions with machine learning models, especially state-of-the-art deep learning models, which have shown better performance in the general domain of time series forecasting. They are also able to handle non-stationary data and complex patterns, which is a common factor of the two datasets used in this work, and a general characteristic of workload traces. Inside these types of models, for a proper demonstration of the performance of the proposed contributions and to check whether they apply across diverse model architectures, we test them in two widely recognized machine learning algorithms, Random Forests (RF) and Support Vector Machines (SVM), and two state-of-the-art deep learning methods from the general domain of time series forecasting, SCINet and Informer, introduced in Section 1.3. The SCINet model stacks multiple convolutional blocks that share latent information and split the time series between odd and even observations for extracting more diverse features. On the other hand, the Informer model is an adaptation of the original Transformer model [45] that adapts the architecture to the time series forecasting domain, tackling, between others, the quadratic time and memory usage. In addition, we include our handcrafted model that we named as Temporal Inception (TempInc). It consists of three convolutional blocks, where each of them gets a different time perspective of the time series, taking the full series, half and a third of it respectively (check Appendix A for a full description of the model). Lastly, we also include the DLinear model from Ailing Zeng et al. [51]. A simple deep learning model based on two fully connected layers for representing the trend and seasonality. It will be useful for checking whether powerful deep learning models are really needed in this scope.

The implementations of the SCINet, Informer and DLinear models are taken from the original code repositories and slightly adapted to the use case, whereas, the implementations for the RF and SVM algorithms come from the *scikit-learn* package [65]. For parametrizing the machine learning algorithms we have opted to use the Bayesian search of the *scikit-optimize* package [66], whereas a simple grid search is employed instead for the case of the deep learning models. Check Appendix B for an extensive description of the specific search hyper-parameters and values.

4.1.4. Evaluation

In the proposed evaluation, we use the values 250, 2, 0.2, 0.5, 4 for the specified constants $\gamma, \sigma, \alpha, \beta, \tau$. With these values, between others, we allow a greater margin over the real usage than under it when predicting a spike, penalizing by a greater margin possible resource starvation problems.

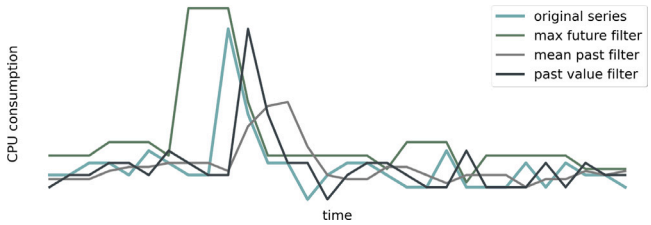


Fig. 5. Example of the three different forecasts in a sliced trace from the Alibaba dataset.

4.1.5. Infrastructure

All experiments are performed in one of the accelerated clusters of MareNostrum IV from the Barcelona Supercomputing Center (BSC). The corresponding specification of every node of this cluster is 2x IBM Power9 8335-GTH @ 2.4 GHz (20 cores, 4 threads/core), 512 GB of RAM and 4 NVIDIA V100 GPU with 16 GB RAM.

4.2. Motivation for new evaluation

In this first set of experiments, we show empirical results of why the new proposed evaluation is required to properly study the methods in the regions that contain sudden increases of consumption. As stated in Section 1.1, other average metrics, largely used in the literature, do not give proper insights. To that end, we create three distinct forecasts that have different performances in the detection of increases of consumption, and evaluate them with the proposed evaluation and the MSE metric, like the main representative of the literature evaluations.

We create the three forecasts with simple filters, *max future filter*, *mean past filter* and *past value filter*, respectively. The first filter, *max future filter*, represents our ideal forecast, in line with the right example of Fig. 1, it is able to predict all spikes with anticipation and it never forecasts under the real usage. This filter assigns the maximum value of the three future samples and elevates the final value by a factor of 10%. It is important to mention that this filter cannot be applied in a real case scenario, due to the usage of future information. We just employ it as a theoretical benchmark to compare the performance of the evaluation metrics in the ideal case. On the other hand, the two other filters represent the left cases of Fig. 1. They do not correctly predict the spikes of consumption, forecasting them after they have occurred or with an incorrect magnitude. The *mean past filter* assigns the mean value of the three antecedent samples, whereas the *past value filter* only assigns the value of the past sample. Fig. 5 shows the behaviour of the three forecasts in an example trace from the Alibaba dataset.

Table 2 presents the results for the MSE metric and the proposed evaluation when applying the three filters to all traces of both datasets. The filters *mean past filter* and *past value filter* obtain the best MSE score, even though they have wrong behaviours during increases of consumption. As we can see in the metric *spikes-rec*, they do not correctly predict the big part of spikes that appear in the datasets. However, the *max future filter* while obtaining the lowest MSE score, has an almost perfect prediction of increases of consumption. As expected, the present is at the cost of a larger over-provisioning as can be check in the corresponding metric. If we chose a model based only on the MSE metric, we would most likely end up with undesirable results.

With all the above, we can say that average metrics like MSE do not give proper insights on the behaviour of the models in regions that contain sudden increases of consumption. The proposed evaluation gives better information in regard to these regions without leaving aside a general study of the over/under-provisioning of the forecasted traces. In that way, the final user can better decide which is the desired trade-off between the two aspects.

Table 2
Results of the three dummy filters in all traces of the Alibaba and Google datasets.

Dataset	Type	MSE ↓	Over-prov ↓	Under-prov ↓	Spikes-prec ↑	Spikes-rec ↑
Alibaba	max future filter	0.5413	3.1661E+14	**0.0000**	0.9958	**0.9987**
	mean past filter	**0.3121**	1.1694E+14	0.0248	0.6175	0.0606
	past value filter	0.4081	**1.1292E+13**	0.0314	**0.9987**	0.1467
Google	max future filter	1.3848	1.1305	**0.0000**	0.9995	**1.0000**
	mean past filter	**0.7595**	**0.1927**	0.0270	0.8686	0.0600
	past value filter	1.0568	0.4094	0.0377	**1.0000**	0.2021

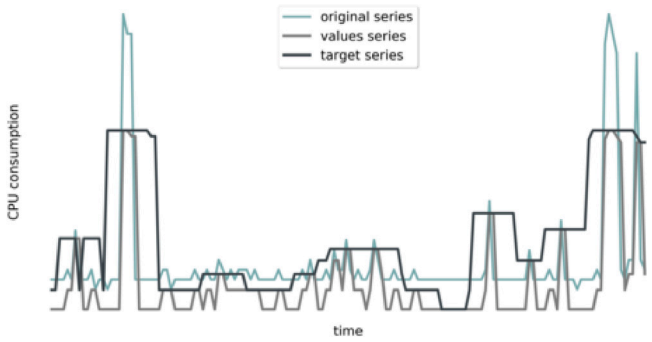


Fig. 6. Example of the transformations that are applied in our experiments when using the distinction between series alteration. This example comes from the CPU consumption of a workload in the Alibaba dataset. The transformations that are applied to create the *values series* consist of a log transformation and a maximum filter with past samples (window equal to 2). On the other hand, the transformations that are applied to create the *target series* consist of a log transformation, a maximum filter with future samples (window equal to 5) and a maximum filter with past samples (window equal to 7).

4.3. Forecasting output alteration

4.3.1. Distinction between series

Firstly, we compare the first proposed strategy to alterate the forecasting output with the standard behaviour in the literature. The standard behaviour consists of using the original time series as the input and output of the forecasting method in a regression or classification manner, whether a discretization step is employed. In contrast, the first proposed alteration makes a distinction between the input and output in two different series, named as *values* and *target*, that contain a set of transformations that better fit the desired goal. In line with our contributions, we decide to change the goal of the training, and thus, the *target series*, to anticipate spikes and, in addition, maintain the increased usage for some time after the spike has occurred for accommodating possible similar future spikes. Of course, this is at the cost of a larger over-provision, but this over-provision should be located only around spikes of usage. Nevertheless, notice that we are only testing one configuration from all possible transformations, others can be used instead that better fit the real case scenario, maybe reducing the margin of anticipation for obtaining lower over-provision values. In regard to the *values series*, which consists of the input of the algorithms, we apply some transformations that have given good results in an initial parameter search. Fig. 6 shows the employed transformations in an example trace. It is important to remark that for testing the alteration strategy we, as well, discretize the resulting *target series* to simplify the learning.

Table 3 exposes the results for the standard behaviour in regression (RBaseline), the standard behaviour in classification (CBaseline) and the proposed distinction between series (Proposed). The results appear segmented by method and dataset. We are using the proposed evaluation, including the F1 score metric as a unique result for the spikes-precision and spikes-recall metrics. Overall, the table shows that the proposed alteration strategy has a clear and substantial improvement in the handling of spikes over the baselines (RBaseline and CBaseline). It holds the best results in the F1 metric for all models and datasets, being the SVM method in the Google dataset the only exception. This improvement in the F1 metric is primarily attributed to the model's enhanced ability to predict a greater number of spikes compared to the baselines, as indicated by the increase in the spikes-recall metric. Although, this increase in the detection of spikes is, for some cases, accompanied by a slight decrease in the metric spikes-precision. The proposed approach is capable of predicting a bigger quantity of spikes at the cost of occasionally mistakenly predicting spikes that will not occur. As expected, this improvement in the handling of spikes comes with a reduction of the under-provisioning, but with a substantial increase in the over-provisioning. This is due to the maximum filters applied to the *target series*, which try to avoid mainly resource starvation problems.

The behaviour of the proposed approach and the baselines can be better visualized in Fig. 7, where we can find examples of the forecasts for the two datasets. We can validate the same conclusions extracted from the metric results, *i.e.*, the proposed approach better handles increases of consumption, by predicting a significant greater number of spikes over the baseline, at the cost of a larger over-provisioning. Nevertheless, as we can see in the figure, this increased over-provisioning is not distributed in the same proportion across all the length of the example traces, it is only located in regions that are close to increases of consumption, usually before, during and after they occur. This not only gives an allocation of resources that can accommodate possible subsequent spikes, but it enables the option to anticipate future spikes, sometimes with a great margin of time. The latter characteristic is an important aspect for the proper allocation of resources in a system, and it is only present in the proposed approach. The baselines are unable to forecast spikes more than one unit of time in the future.

Apart from this, it is interesting to point out the improvement in the spike metrics when using powerful deep learning models. Simple methods, like the Dlinear or SVM models, are not able to accomplish the same outcomes. Seems that the time series come with complex patterns and non-linearities in the level, trend and seasonality that demand a deep understanding of the data. Unfortunately, this comes at the cost of a longer training time, *i.e.* the Temporal Inception and Informer models take around 5 h to train in the Alibaba dataset. Nevertheless, the inference time is around 0.05 ms per sample when working with batch sizes of size 512, which is enough margin to predict windows of 5 min in advance.

4.3.2. Loss function modification

Secondly, we test the second alteration strategy that adapts the outputs from the forecasting through the ground truth values of the loss function of deep learning models. We compare the difference in the forecastings between training the model with this second proposed strategy and training it with the MSE and CE loss functions, that constitute the baselines in this subsection. Before applying the CE loss, we apply a discretization step in the output values. We analyse the results from the model that obtained the best F1-score in the previous Section 4.3.1, *i.e.*, TempInc. Four different altered ground truth variants of our proposed strategy are tested: *single*, *binary*, *proportional* and *exponential*. In the *single* variant, only the correct interval takes the full value (1), whereas, in the *binary* version, all intervals over the correct one receive the full value (1) and all other intervals receive the empty

Table 3
Results of the proposed forecasting output alteration (Proposed) in all models and both datasets alongside the results for the standard behaviours in the literature for forecasting: regression (RBaseline) and classification (CBaseline).

Dataset	Algorithm	Type	Over-prov ↓	Under-prov ↓	Spikes-prec ↑	Spikes-rec ↑	Spikes-F1 ↑
Alibaba	RF	RBaseline	**1.7923E+12**	0.0173	**0.8039**	0.0663	0.0888
		CBaseline	1.9181E+12	0.0215	0.5671	0.1168	0.1139
		Proposed	4.8636E+12	**0.0102**	0.6709	**0.2654**	**0.3304**
	SVM	RBaseline	**7.8987E+11**	0.0249	**0.8251**	0.0306	0.0385
		CBaseline	1.2890E+13	0.0571	0.2774	0.1313	0.0786
		Proposed	1.9454E+13	**0.0194**	0.2833	**0.2352**	**0.1489**
	DLinear	RBaseline	**6.4819E+11**	0.0249	**0.6656**	0.0918	0.1385
		CBaseline	1.9181E+12	0.0234	0.5640	0.0909	0.1033
		Proposed	4.8636E+12	**0.0095**	0.5717	**0.3012**	**0.3431**
	TempInc	RBaseline	4.9336E+12	0.0173	**0.7224**	0.1577	0.2220
		CBaseline	**1.9181E+12**	0.0158	0.7092	0.2117	0.2745
		Proposed	4.8636E+12	**0.0047**	0.7213	**0.6335**	**0.6378**
	SCINet	RBaseline	**8.7268E+10**	0.0322	0.7014	0.0742	0.1006
		CBaseline	1.9181E+12	0.0184	0.6185	0.1298	0.1413
		Proposed	4.8636E+12	**0.0076**	**0.7597**	**0.2955**	**0.3902**
	Informer	RBaseline	4.4108E+12	0.0198	0.8316	0.0579	0.0859
		CBaseline	**1.9181E+12**	0.0183	0.6601	0.1347	0.1581
		Proposed	4.8636E+12	**0.0076**	**0.8787**	**0.3476**	**0.4624**
Google	RF	RBaseline	**2.7032E+16**	0.0162	**0.9502**	0.1648	0.2480
		CBaseline	4.3642E+16	0.0079	0.5277	0.4140	0.3311
		Proposed	1.2232E+17	**0.0018**	0.6380	**0.6695**	**0.5777**
	SVM	RBaseline	**5.0567E+16**	0.0217	**0.9269**	0.0509	0.0787
		CBaseline	7.6060E+16	0.0113	0.3070	0.3691	**0.2017**
		Proposed	1.0476E+17	**0.0054**	0.1428	**0.5291**	0.1317
	DLinear	RBaseline	**7.1316E+16**	0.0165	**0.8559**	0.1764	0.2606
		CBaseline	1.9846E+17	0.0071	0.5747	0.4322	0.4027
		Proposed	6.4843E+17	**0.0017**	0.7010	**0.6969**	**0.6617**
	TempInc	RBaseline	4.0283E+17	0.0143	**0.9358**	0.2361	0.3451
		CBaseline	**2.4622E+17**	0.0065	0.4021	0.5176	0.3358
		Proposed	3.8081E+18	**0.0014**	0.9087	**0.7414**	**0.7973**
	SCINet	RBaseline	8.4521E+16	0.0166	0.8390	0.1751	0.2480
		CBaseline	**6.1426E+16**	0.0069	0.4608	0.4750	0.3477
		Proposed	1.6249E+18	**0.0015**	**0.8764**	**0.7295**	**0.7724**
	Informer	RBaseline	1.0973E+17	0.0175	**0.9682**	0.1360	0.2146
		CBaseline	**4.1801E+16**	0.0070	0.4249	0.4439	0.3242
		Proposed	1.3126E+18	**0.0016**	0.9286	**0.7092**	**0.7848**

value (0). Similarly, the *proportional* variant gives the empty value to all intervals below the ground truth, and gives, to all intervals above, a value linearly proportional to the proximity to the correct interval, whereas, in the *exponential* variant, a decaying exponential function is used instead.

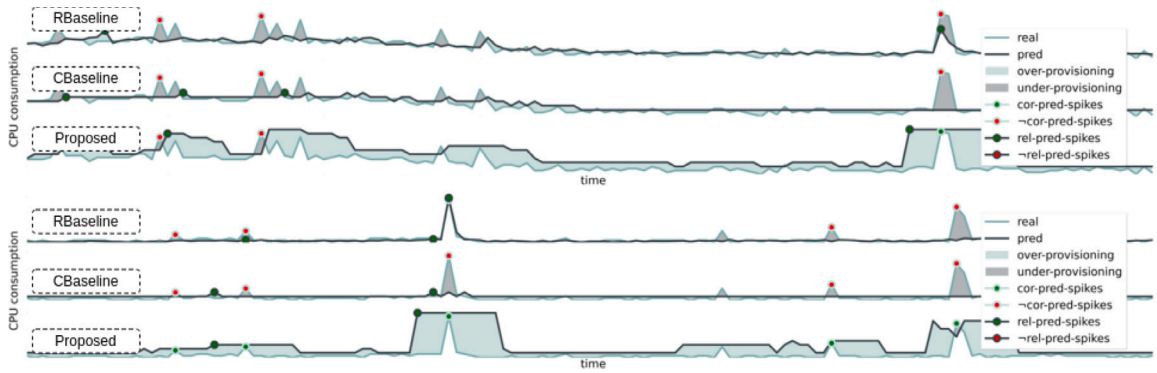
Table 4 presents the results for the proposed evaluation in the two datasets. The results for the altered variants correspond to the experiment that obtained the best spikes-F1 score when testing different values for the α constant and different magnitudes for the exponential and proportional functions with a temperature T equal to 0.1. As expected, the *binary* variant ends up with poor results, it modifies the learning of the model to predict far over the real usage, without any real connection to the real usage. Nevertheless, the rest of variants show better general results with an improved management of the spikes over the baselines, with a significant increase in the F1 score. In regard to the over-provisioning, the proposed variants obtain similar results for the Alibaba dataset and a significant increase in the Google dataset. Surprisingly, the *single* version, which only assigns the full value to the correct interval, in a similar fashion to the behaviour of the baselines, obtains very good scores in the spikes-F1 metric compared to the *proportional* and *exponential* variants. This brings concerns about the usefulness of the altered ground truth for our specific use case of cloud provisioning.

What is clear is that modifying the ground truth values in the loss function has a clear impact on the resulting forecasts. Fig. 8 shows the behaviour of the *exponential* variant when using different magnitudes for the function slope. The slope difference transforms the final outcome of the learning, predicting with a greater distance over the real usage when the slope decreases. The results from the medium slope are remarkable, having a dynamic behaviour that maintains a low-provision during moments of little variance, but captures spikes of consumption once they start to appear at the cost of a larger over-provisioning.

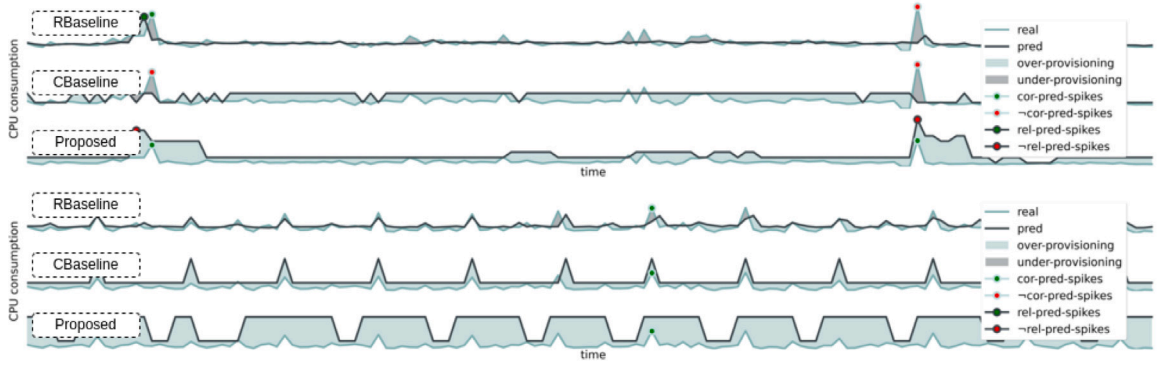
4.4. Explainability

For the last set of experiments, we study the latent behaviour of the models, trying to gain insight on the reasons behind their predictions. Specifically, with the goal of obtaining a general understanding of reasons behind the predictions of sudden increases of consumption. The obtention of explainable results has been a growing request in the domain of Artificial Intelligence since the apparition of complex deep learning models, with the greatest number of contributions in the fields of image and language data. Still, time series data has not been left behind, plenty of works apply explainability techniques into this kind of data [67]. The interpretation of the models' results can generate more trust in their outcomes and can help to locate or better understand patterns of the original data.

In this case, we analyse the results from the model that obtained the best F1-score in Section 4.3.1, *i.e.*, TempInc. To that end, we opt for using the SHAP extension of the DeepLift algorithm [68,69]. It is a propagation-based approach that works with general neural network structures that only requires a single backward pass. The algorithm extracts the degree of contribution to generate a specific output from every section of the input (attributions). The parts that end up with a greater contribution value correspond to the areas that the model takes



(a) Two sliced traces from the Alibaba dataset.



(b) Two sliced traces from the Google dataset.

Fig. 7. Two sliced traces from the Alibaba dataset.

Table 4
Results of the proposed variants of the DdCE loss function alongside the MSE and CE alternatives. All results are computed with the Temporal Inception model, which obtained the best F1 score for both datasets in the prior set of experiments.

Dataset	Loss function	Over-prov ↓	Under-prov ↓	Spikes-prec ↑	Spikes-rec ↑	Spikes-F1 ↑
Alibaba	MSE	4.9336E+12	0.0173	0.7224	0.1577	0.2220
	CE	1.9181E+12	0.0158	0.7092	0.2117	0.2745
	Altered - single	1.9181E+12	0.0144	**0.7342**	0.3000	0.3868
	Altered - binary	1.9181E+12	0.0150	0.5122	0.1811	0.1798
	Altered - proportional	1.9181E+12	**0.0114**	0.6437	0.3401	0.3940
	Altered - exponential	1.9181E+12	0.0132	0.6287	**0.3454**	**0.4005**
Google	MSE	4.0283E+17	0.0143	**0.9358**	0.2361	0.3451
	CE	**2.4622E+17**	0.0065	0.4021	0.5176	0.3358
	Altered - single	5.5718E+17	0.0065	0.5479	0.5052	**0.4455**
	Altered - binary	3.0856E+17	0.0058	0.4151	0.5206	0.3307
	Altered - proportional	5.9307E+17	**0.0057**	0.4833	**0.5379**	0.3854
	Altered - exponential	6.9480E+17	0.0063	0.4654	0.5225	0.3849

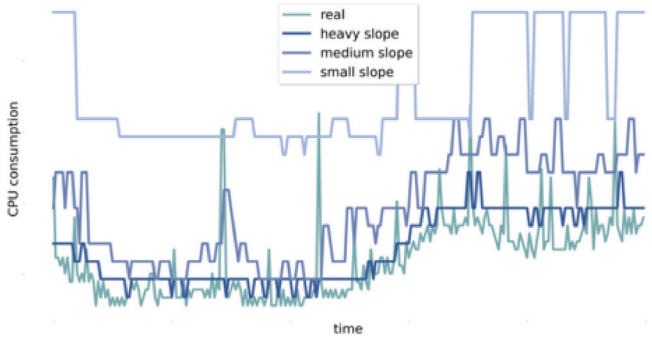
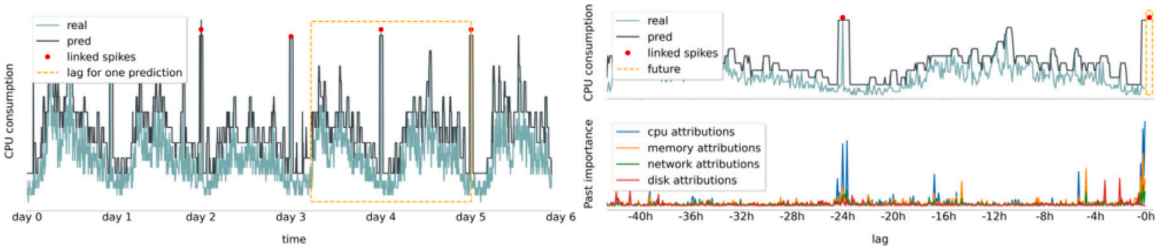


Fig. 8. Example of the behaviour of the altered *exponential* variant with different slope magnitudes in a sliced trace from the Alibaba dataset. A max past filter with window 3 was employed in the medium and heavy configurations as a post-processing step to decrease the instability of the forecasts.

more into account for generating the specific output. As an example, Fig. 9.a shows a complete trace from the Alibaba dataset that contains a clear seasonal pattern with spikes at the start of every day. In order to predict the time point corresponding to the last spike of the trace, the model takes a window of the past 42 h as input. This window is depicted on Fig. 9.b alongside the value for the attributions of each time point. As it can be seen, the model gives special attention to the values located close to the present and the ones located 24 h prior, corresponding to the previous spike of usage.

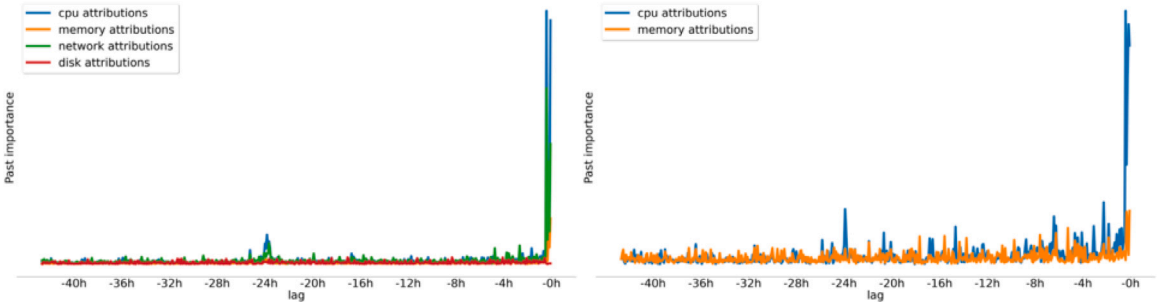
If we average the attributions for all samples of both datasets, we obtain the charts exposed in Fig. 10. It is clear that the model retrieves the majority of information from the samples that are near the present. Presumably, this is in order to better understand the current trend and level from the series, and maybe to extract patterns in the recent observations that the model has learnt from the overall dataset. Additionally, there are other points in the past that have some reasonable importance. In the case of Alibaba traces, a explicit daily seasonality can be noticed, as in the specific example of Fig. 9. Likewise, in this dataset, we can conclude that the big part of past



(a) CPU consumption of a complete trace. The red dots remark a set of spikes that repeat at the beginning of every day. The orange rectangle corresponds to the window that the model uses as input to predict the last labeled spike.

(b) Sliced trace corresponding to the window used to predict the last labeled spike of the chart on the left. On top, the original trace (*real*) and the model forecasts (*pred*). Below, the attributions corresponding to every past position categorized by feature type.

Fig. 9. Explainability attributions in an trace from the Alibaba dataset that contains a strong seasonality pattern.



(a) Alibaba

(b) Google

Fig. 10. Averaged attributions for all forecasts of every sample of both datasets. The attributions are divided by type of feature. Notice that the Google dataset does not contain network or disk features.

samples are not important for future predictions. To the contrary, the Google dataset contains a more diverse set of attribution values, with significant importance in many different past positions, not only 24 h prior to the prediction. Apart from this, taking a look to the types of features, the two figures show that the model collects the large part of information from CPU features, which is logical since the scope and goal of all the experiments in this paper are to forecast CPU consumption. Nevertheless, other features, like network and memory, have some relative importance as well and should not be ignored.

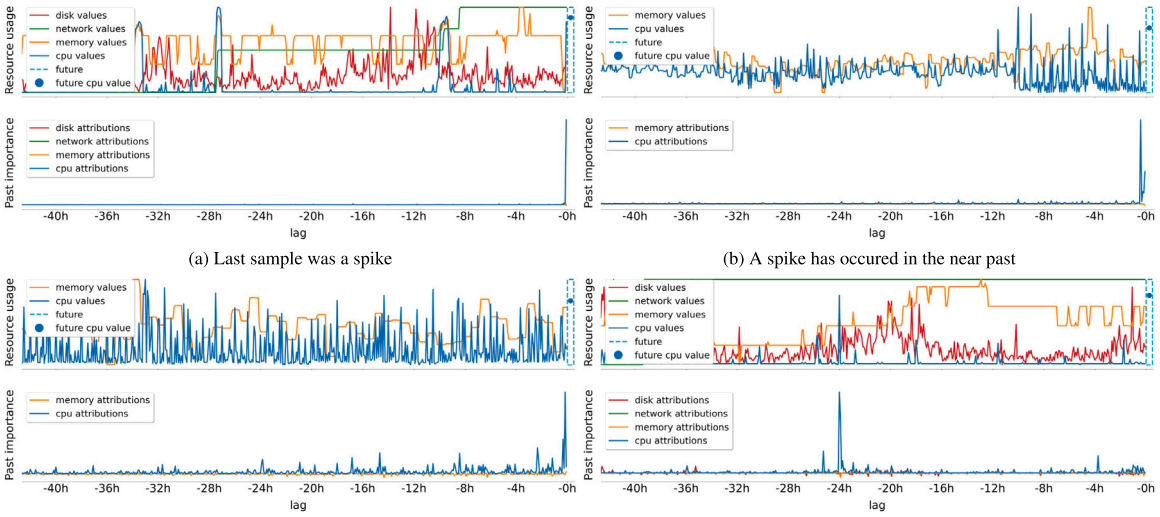
Averaging the full set of attributions it is helpful to visualize general and common patterns, but it takes away the opportunity to extract concrete behaviours. As stated at the beginning of the section, our main goal is to analyse the reasons behind the model predictions of spikes, which is not only interesting to explain the model decisions but to better understand the patterns in the traces that can stimulate a sudden increase. In that direction, one of the best examples in the literature is the work from Avanti Shrikumar et al. [70]. With a set of preprocessing, filtering and clustering steps they were able to find specific and clear patterns for attributions in genomic data. Although a full process of pattern recognition is outside the scope of this work, we perform a brief processing of the attributions before increases of consumption in order to have a general idea of the behaviour of the resulting models of this work.

Instead of treating and analysing the full set of spikes of consumption, we select the ones that were correctly predicted and that suppose a higher increase in regard to the previous set of samples. In this way, we simplify the task of interpreting the results and prioritize the spikes that have a higher impact on the resource management of a cluster. We choose the spikes that represent the right tail of their magnitude distribution, that is, the heaviest 2000 and 7000 spikes from the Alibaba and Google datasets, respectively. After this selection, we clusterize their attributions with the KMeans algorithm and visualize their outcome with the t-SNE technique. In order to simplify the clusterization task, instead of working with the full vector of attributions for every spike, each vector of attributions is temporarily reduced with a mean filter that averages the values in windows of 5, and a PCA reduction is applied to reduce the variance to a few number of features.

Despite being regression data, we are able to extract four common reasons in both datasets that the model uses to anticipate a future spike. These four patterns are exposed in Fig. 11 and could be described as (a) last sample was a spike (b) a spike has occurred in the near past (c) there is huge variability in the near past (d) there is a clear daily seasonality. Future work is required to analyse all results and pattern variations, digging further in more specific patterns present in each dataset. The model presents both reactive and predictive behaviours, reacts to increases of consumption in the near past, but also is able to understand and forecast seasonal patterns. Notice that we fed the model with *target series* that make the forecasts maintain a big consumption value after a spike has occurred, which could be changed if other intentions were desired.

5. Discussion

From the previous results in Tables 3 and 4, we can see that the two proposed strategies improve the forecasting of sudden increases of consumption in resource traces for the vast majority of tested algorithms. They obtain an overall improvement in the spikes-F1 metric, which assesses whether the algorithm is able to correctly anticipate spikes with the right height (true positives) without over predicting spikes that will never occur (false positives). In line with our initial goal exposed in the right example of Fig. 1 or the tested configuration of the *target series* in Fig. 6, this improvement in the handling of spikes is obtained at the cost of a larger over-provision. Nevertheless, this over-provision is located only around regions that contain sudden increases of consumption, anticipating some of them with a great margin and maintaining the increased resource usage to accommodate future similar spikes. This can be seen in Figs. 7 and 8, that visually show the behaviour of the predictions for traces of the Alibaba and Google datasets. As aforementioned, if a lower over-provision is desired at the cost of decreasing the performance in the prediction of spikes,



(a) Last sample was a spike

(b) A spike has occured in the near past

(c) There is a huge variability in the near past

(d) There is a clear daily seasonality

Fig. 11. Some common patterns found in both datasets that estimulate a future spike of CPU consumption.

both strategies can be slightly modified to obtain predictions in this new direction.

If we compare both strategies, the first strategy, based on the distinction between series, comes on top. It is able to predict more increases of consumption with fewer cases of false positives. In addition, it allows anticipating some spikes by a considerable margin of time, as can be seen in Fig. 7. Nevertheless, the second strategy, consisting of modifying the ground truth directly in the loss function of deep learning models, leaves more freedom for defining the desired output distribution. Future work should involve a combination of the two strategies to check if the final results can be improved. Similarly, more experiments with other types of resources would be beneficial, to check whether these results can be reproduced with memory, network or disk consumption.

It is important to remark that both strategies can be applied to other use cases in cloud provisioning or to any other field inside the domain of time series forecasting. The ground truth in the two strategies can be modified to comply with any desired objective. In addition, thanks to having low latencies, these strategies can be used in online-systems to adjust sudden spikes of demand. When using the Informer model, the inference time is only around 0.05 ms per sample.

Lastly, even though we presented some insights on the reasons behind the predictions in Section 4.4, there is plenty of more work to do. It would be very useful to extract a complete list, or a big part, of the patterns that the models recognize in order to forecast future increases of consumption. It would allow the creation of a simple light-weighted rule system with similar performance. Nevertheless, unfortunately, the prior seems quite improbable. Having a closer look at the table results, the most complex models *i.e.*, TemporalInception and Informer obtain considerable better scores than the simpler Dlinear or SVM versions, implying that the task at hand probably is complex enough to not be possible a reduction to a rule system. Also, the present reflects the importance of using state-of-the-art models in cloud provisioning scenarios. They require more resources, but the difference in performance is notorious, and the inference time, as aforementioned, is sufficiently good enough for an online production system.

6. Conclusion

In this work, we presented two distinct strategies to modify the behaviour of forecasting algorithms to better accommodate their results to the cloud provisioning field. They let the user decide the trade-off between correctly predicting sudden increases of consumption and reducing the amount of over-provisioning. We tested these strategies with the focus on improving the handling of spikes of usage, trying to anticipate them with a good margin of time and correctly predicting their height. The obtained predictions can lead to a better management of the resources in a cluster, using the resources of the computing nodes in a more effective manner. In addition, we proposed a new evaluation methodology that gives better insights in the quality of the forecasts in these kinds of scenarios, and employed explainability techniques to better understand the reasons behind the algorithms' predictions.

CRediT authorship contribution statement

Ferran Agulló: Writing – original draft, Visualization, Software, Investigation, Data curation, Conceptualization. **Alberto Gutierrez-Torre:** Writing – review & editing, Supervision, Project administration. **Jordi Torres:** Writing – review & editing, Supervision, Funding acquisition. **Josep Ll. Berral:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Ferran Agullo Lopez reports financial support was provided by Spain Ministry of Science and Innovation CEX2021-001148-S-20-3. Ferran Agullo Lopez reports financial support was provided by EU-HORIZON GA.101092646 (CloudSkin). Ferran AgulloLopez reports financial support was provided by EU-HORIZON MSCA GA.101086248(CloudStars). Josep Ll. Berral reports financial support was provided by Spain Ministry of Science and Innovation PID2021-126248OB-I00. Josep Ll. Berral reports financial support was provided by Spanish Research State Agency MCIN-AEI-10.13039-501100011033-FEDER. Josep Ll. Berral reports financial support was provided by Generalitat de Catalunya (AGAUR) 2021-SGR-00478. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work has been supported by the EU's Horizon research and innovation programme under grant agreements HORIZON GA.101092646

(CloudSkin), HORIZON MSCA GA.101086248 (CloudStars), and by the Spanish Ministry of Science (MICINN), the Research State Agency (AEI) and European Regional Development Funds (ERDF/FEDER) under grant agreements PID-2021-126248OB-I00, MCIN/AEI/10.13039/501100011033, UE (DALEST), Severo Ochoa Center of Excellence CEX-2021-001148-S-20-3 and the Generalitat de Catalunya (AGAUR), Spain 2021-SGR-00478 (CROMAI). During the preparation of this work the authors used BSC gepeto AI in order to improve the writing of some sentences. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Appendix A. Hyper-parametrization of methods

This appendix includes the hyper-parameter search specification, applied for extracting the results of all rows from Table 3 alongside some more specific information about their implementation. The maximum number of iterations or epochs in the training of the models have been fixed to not surpass a time limit of 2 days per experiment (limitation imposed by the computing cluster being employed).

Random Forest (RF): For regression scenarios we use the model *RandomForestRegressor* from *scikit-learn* with the *gini* loss, whereas, we use *RandomForestRegressor* for the classification scenarios with the *mse* loss. We perform a Bayesian search with *scikit-optimize* for the parameters *n_estimators*, *min_samples_leaf*, *max_depth* and *max_features* with values *[8, 100, "uniform"]*, *[1, 512, "uniform"]*, *[4, 4096, "uniform"]* and *[2, 8, "uniform"]* respectively. A different search is done for the lag size values 32, 128, 512. Because of memory issues, only the experiments with lag size equal to 32 are performed with full input features. The rest is done with only the target feature as input.

Support Vector Machine (SVM): For regression scenarios we use the model *LinearSVR* from *scikit-learn* with the *squared_epsilon_insensitive* loss, whereas, we use *LinearSVC* for the classification scenarios with the *squared_hinge* loss. We perform a Bayesian search with *scikit-optimize* for the parameters *C* and *epsilon* with values *[0.0001, 1000.0, "uniform"]* and *[0.0, 4.0, "uniform"]* respectively (the parameter epsilon is only used in the regression scenario). A different search is done for the lag size values 32, 128, 512. Because of memory issues, only the experiments with lag size equal to 32 are performed with full input features. The others are done with only the target feature as input.

DLinear: We add a fully connected layer at the start of the original implementation to reduce the input number of features. We perform a grid search for the parameters *lag size* and *learning rate* with values *[32, 128, 512]* and *[0.1, 0.01, 0.001]* respectively.

Temporal Inception (TempInc): We perform a grid search for the parameters *lag size* and *learning rate* with values *[32, 128, 512]* and *[0.1, 0.01, 0.001]* respectively.

SCINet: We add a fully connected layer at the end of the original implementation to reduce the output number of features to only one (regression scenario) or the number of intervals (classification scenario). We perform a grid search for the parameters *lag size*, *learning rate*, *hid_size*, *num_levels* and *dropout* with values *[32, 128, 512]*, *[0.1, 0.01, 0.001]*, *[1, 2]*, *[1, 2]* and *[0, 0.5]* respectively.

Informer: We add a fully connected layer at the end of the original implementation to reduce the output number of features to only one (regression scenario) or the number of intervals (classification scenario). We perform a grid search for the parameters *lag size*, *learning rate*, *d_model/d_ff*, *n_heads* and *e_layers/d_layers* with values *[32, 128, 512]*, *[0.1, 0.01, 0.001]*, *[32, 128]*, *[2, 4]* and *[2, 4]* respectively. In the case of Google, we include to the grid search the parameter *dropout* with values *[0, 0.5]*. This is due to the found convergence problems in this dataset.

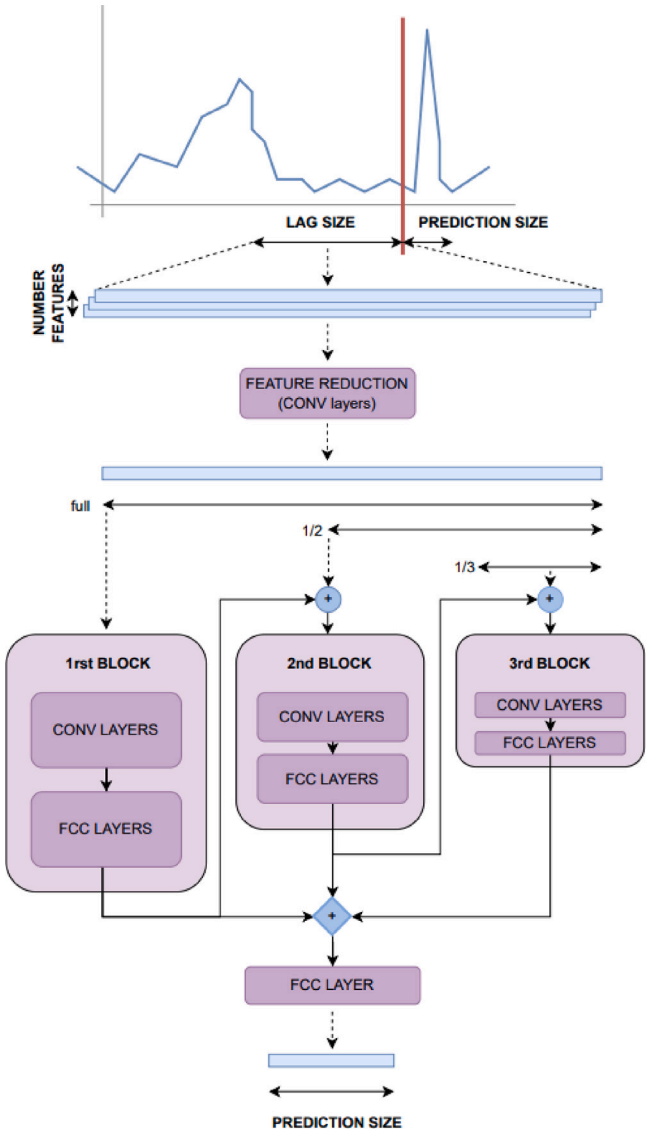


Fig. B.12. Diagram of the architecture of the Temporal Inception model (TempInc).

Appendix B. Temporal inception architecture

Fig. B.12 shows a diagram of the Temporal Inception model (TempInc), our own craft model that appears in the experiments section. Check the specified code repository for its implementation. Notice that it is an experimental model, it is not much modifiable by hyperparameters.

Data availability

The datasets used are largely explained in the manuscript, and they can be accessed and downloaded from the cited links.

References

[1] M. Alam, S. Mustajab, M. Shahid, F. Ahmad, Cloud computing: Architecture, vision, challenges, opportunities, and emerging trends, in: 2023 International Conference on Computing, Communication, and Intelligent Systems, ICCCIS, 2023, pp. 829–834, http://dx.doi.org/10.1109/ICCCIS60361.2023.10425507.

[2] F. Brocklehurst, International review of energy efficiency in data centres, 2021, Ballarat Consulting for the Australian Department of Industry, Science, Energy and Resources.

[3] M. Koot, F. Wijnhoven, Usage impact on data center electricity needs: A system dynamic forecasting model, Appl. Energy 291 (2021) 116798, http://dx.doi.org/10.1016/j.apenergy.2021.116798, URL https://www.sciencedirect.com/science/article/pii/S0306261921003019.

[4] D. Saxena, A.K. Singh, Workload forecasting and resource management models based on machine learning for cloud computing environments, 2021, arXiv preprint arXiv:2106.15112.

[5] M. Alam, M. Shahid, S. Mustajab, A security driven energy efficient workflow allocation algorithm under deadline constraints for cloud computing, in: 2023 4th International Conference on Data Analytics for Business and Industry, ICDABI, 2023, pp. 331–336, http://dx.doi.org/10.1109/ICDABI60145.2023.10629337.

[6] M. Masdari, A. Khoshnevis, A survey and classification of the workload forecasting methods in cloud computing, Clust. Comput. 23 (4) (2020) 2399–2424, http://dx.doi.org/10.1007/s10586-019-03010-3.

[7] T. Khan, W. Tian, S. Ilager, R. Buyya, Workload forecasting and energy state estimation in cloud data centres: ML-centric approach, Future Gener. Comput. Syst. 128 (2022) 320–332, http://dx.doi.org/10.1016/j.future.2021.10.019, URL https://www.sciencedirect.com/science/article/pii/S0167739X21004155.

[8] D. Janardhanan, E. Barrett, CPU workload forecasting of machines in data centers using LSTM recurrent neural networks and ARIMA models, in: 2017 12th International Conference for Internet Technology and Secured Transactions, ICITST, 2017, pp. 55–60, http://dx.doi.org/10.23919/ICITST.2017.8356346.

[9] S. Ouhame, Y. Hadi, A. Ullah, An efficient forecasting approach for resource utilization in cloud data center using CNN-LSTM model, Neural Comput. Appl. 33 (16) (2021) 10043–10055, http://dx.doi.org/10.1007/s00521-021-05770-9.

[10] D. Saxena, A.K. Singh, Auto-adaptive learning-based workload forecasting in dynamic cloud environment, Int. J. Comput. Appl. 44 (6) (2022) 541–551, http://dx.doi.org/10.1080/1206212X.2020.1830245.

[11] A. Verma, L. Pedrosa, M.R. Korupolu, D. Oppenheimer, E. Tune, J. Wilkes, Large-scale cluster management at Google with Borg, in: Proceedings of the European Conference on Computer Systems, EuroSys, Bordeaux, France, 2015, pp. 1–17.

[12] M. Tirmazi, A. Barker, N. Deng, M.E. Haque, Z.G. Qin, S. Hand, M. Harchol-Balter, J. Wilkes, Borg: the next generation, in: Proceedings of the Fifteenth European Conference on Computer Systems, 2020, pp. 1–14.

[13] C. Jiang, G. Han, J. Lin, G. Jia, W. Shi, J. Wan, Characteristics of co-allocated online services and batch jobs in internet data centers: A case study from Alibaba cloud, IEEE Access 7 (2019) 22495–22508, http://dx.doi.org/10.1109/ACCESS.2019.2897898.

[14] C. Lu, K. Ye, G. Xu, C.-Z. Xu, T. Bai, Imbalance in the cloud: An analysis on Alibaba cluster trace, in: 2017 IEEE International Conference on Big Data, Big Data, 2017, pp. 2884–2892, http://dx.doi.org/10.1109/BigData.2017.8258257.

[15] A. Fox, R. Griffith, A. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, et al., Above the Clouds: A Berkeley View of Cloud Computing, Rep. UCB/EECS, 28 (13), Dept. Electrical Eng. and Comput. Sciences, University of California, Berkeley, 2009, p. 2009.

[16] M. Amiri, L. Mohammad-Khanli, Survey on prediction models of applications for resources provisioning in cloud, J. Netw. Comput. Appl. 82 (2017) 93–113.

[17] V. Le Guen, N. Thome, Deep time series forecasting with shape and temporal criteria, IEEE Trans. Pattern Anal. Mach. Intell. 45 (1) (2022) 342–355.

[18] R. Weingärtner, G.B. Bräscher, C.B. Westphall, Cloud resource management: A survey on forecasting and profiling models, J. Netw. Comput. Appl. 47 (2015) 99–106.

[19] D. Saxena, A.K. Singh, Workload forecasting and resource management models based on machine learning for cloud computing environments, 2021, CoRR abs/2106.15112, arXiv:2106.15112, URL https://arxiv.org/abs/2106.15112.

[20] N. Wu, Y. Xie, A survey of machine learning for computer architecture and systems, ACM Comput. Surv. 55 (3) (2022) 1–39.

[21] N. Roy, A. Dubey, A. Gokhale, Efficient autoscaling in the cloud using predictive models for workload forecasting, in: 2011 IEEE 4th International Conference on Cloud Computing, 2011, pp. 500–507, http://dx.doi.org/10.1109/CLOUD.2011.42.

[22] J.L. Berral, D. Buchaca, C. Herron, C. Wang, A. Youssef, Theta-scan: Leveraging behavior-driven forecasting for vertical auto-scaling in container cloud, in: 2021 IEEE 14th International Conference on Cloud Computing, CLOUD, 2021, pp. 404–409, http://dx.doi.org/10.1109/CLOUD53861.2021.00054.

[23] S. Li, Y. Wang, X. Qiu, D. Wang, L. Wang, A workload prediction-based multi-VM provisioning mechanism in cloud computing, in: 2013 15th Asia-Pacific Network Operations and Management Symposium, APNOMS, 2013, pp. 1–6.

[24] Z. Gong, X. Gu, J. Wilkes, PRESS: Predictive elastic ReSource scaling for cloud systems, in: 2010 International Conference on Network and Service Management, 2010, pp. 9–16, http://dx.doi.org/10.1109/CNSM.2010.5691343.

[25] A. Khan, X. Yan, S. Tao, N. Anerousis, Workload characterization and prediction in the cloud: A multiple time series approach, in: 2012 IEEE Network Operations and Management Symposium, 2012, pp. 1287–1294, http://dx.doi.org/10.1109/NOMS.2012.6212065.

[26] K. Cetinski, M.B. Juric, AME-WPC: Advanced model for efficient workload prediction in the cloud, J. Netw. Comput. Appl. 55 (2015) 191–201, http://dx.doi.org/10.1016/j.jnca.2015.06.001, URL https://www.sciencedirect.com/science/article/pii/S1084804515001241.

[27] S.-U.-R. Baig, W. Iqbal, J.L. Berral, A. Erradi, D. Carrera, Adaptive prediction models for data center resources utilization estimation, IEEE Trans. Netw. Serv. Manag. 16 (4) (2019) 1681–1693, http://dx.doi.org/10.1109/TNSM.2019.2932840.

[28] J. Zhu, R. Yang, C. Hu, T. Wo, S. Xue, J. Ouyang, J. Xu, Perph: A workload co-location agent with online performance prediction and resource inference, in: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing, CCGrid, 2021, pp. 176–185, http://dx.doi.org/10.1109/CCGrid51090.2021.00027.

[29] X. Tang, X. Liao, J. Zheng, X. Yang, Energy efficient job scheduling with workload prediction on cloud data center, Clust. Comput. (2018) http://dx.doi.org/10.1007/s10586-018-2154-7.

[30] K. Rzadca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand, et al., Autopilot: workload autoscaling at Google, in: Proceedings of the Fifteenth European Conference on Computer Systems, 2020, pp. 1–16.

[31] F.J. Baldan, S. Ramirez-Gallego, C. Bergmeir, F. Herrera, J.M. Benitez, A forecasting methodology for workload forecasting in cloud systems, IEEE Trans. Cloud Comput. 6 (4) (2018) 929–941, http://dx.doi.org/10.1109/TCC.2016.2586064.

[32] I.K. Kim, W. Wang, Y. Qi, M. Humphrey, Forecasting cloud application workloads with CloudInsight for predictive resource management, IEEE Trans. Cloud Comput. 10 (3) (2022) 1848–1863, http://dx.doi.org/10.1109/TCC.2020.2998017.

[33] B. Liu, Y. Lin, Y. Chen, Quantitative workload analysis and prediction using Google cluster traces, in: 2016 IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS, 2016, pp. 935–940, http://dx.doi.org/10.1109/INFCOMW.2016.7562213.

[34] W. Zhong, Y. Zhuang, J. Sun, J. Gu, A load prediction model for cloud computing using PSO-based weighted wavelet support vector machine, Appl. Intell. 48 (11) (2018) 4072–4083, http://dx.doi.org/10.1007/s10489-018-1194-2.

[35] M. Daraghmeh, A. Agarwal, R. Manzano, M. Zaman, Time series forecasting using facebook prophet for cloud resource management, in: 2021 IEEE International Conference on Communications Workshops, ICC Workshops, 2021, pp. 1–6, http://dx.doi.org/10.1109/ICCWorkshops50388.2021.9473607.

[36] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, R. Bianchini, Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms, in: Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 153–167, http://dx.doi.org/10.1145/3132747.3132772.

[37] G. Shmueli, K. Lichtendahl, Practical Time Series Forecasting with R: A Hands-On Guide, second ed., Practical Analytics, Axelrod Schnall Publishers, 2016, URL https://books.google.es/books?id=SOtgvgAACAAJ.

[38] C. Xiao, J. Huang, W. Wu, Y. Yin, H. Chang, Enhanced-XGB: An online service resource demand forecasting method for colocation data centers, in: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing, CCGrid, 2021, pp. 436–444, http://dx.doi.org/10.1109/CCGrid51090.2021.00053.

[39] J. Kumar, R. Goomer, A.K. Singh, Long short term memory recurrent neural network (LSTM-RNN) based workload forecasting model for cloud datacenters, Procedia Comput. Sci. 125 (2018) 676–682, http://dx.doi.org/10.1016/j.procs.2017.12.087, URL https://www.sciencedirect.com/science/article/pii/S1877050917328557, The 6th International Conference on Smart Computing and Communications.

[40] B. Song, Y. Yu, Y. Zhou, Z. Wang, S. Du, Host load prediction with long short-term memory in cloud computing, J. Supercomput. 74 (2018) 6554–6568.

[41] M. Abouelyazid, Forecasting resource usage in cloud environments using temporal convolutional networks, Appl. Res. Artif. Intell. Cloud Comput. 5 (1) (2022) 179–194.

[42] J. Kumar, A.K. Singh, R. Buyya, Self directed learning based workload forecasting model for cloud resource management, Inform. Sci. 543 (2021) 345–366.

[43] H. Chen, R.A. Rossi, K. Mahadik, S. Kim, H. Eldardiry, Graph deep factors for forecasting with applications to cloud resource allocation, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery; Data Mining, KDD '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 106–116, http://dx.doi.org/10.1145/3447548.3467357.

[44] K. Cho, B. van Merrienboer, D. Bahdanau, Y. Bengio, On the properties of neural machine translation: Encoder-decoder approaches, 2014, CoRR abs/1409.1259, arXiv:1409.1259, URL http://arxiv.org/abs/1409.1259.

[45] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L.u. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017, URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[46] S. Zhou, J. Li, K. Zhang, M. Wen, Q. Guan, An accurate ensemble forecasting approach for highly dynamic cloud workload with VMD and R-transformer, IEEE Access 8 (2020) 115992–116003, http://dx.doi.org/10.1109/ACCESS.2020.3004370.
[47] J. Zhu, W. Bai, J. Zhao, L. Zuo, T. Zhou, K. Li, Variational mode decomposition and sample entropy optimization based transformer framework for cloud resource load prediction, Knowl.-Based Syst. 280 (2023) 111042, http://dx.doi.org/10.1016/j.knosys.2023.111042, URL https://www.sciencedirect.com/science/article/pii/S095070512300792X.
[48] W. Qi, J. Yao, J. Li, W. Wu, Performer: A resource demand forecasting method for data centers, in: C. Yu, J. Zhou, X. Song, Z. Lu (Eds.), Green, Pervasive, and Cloud Computing, Springer International Publishing, Cham, 2023, pp. 204–214.
[49] M. Liu, A. Zeng, Q. Lai, Q. Xu, Time series is a special sequence: Forecasting with sample convolution and interaction, 2021, CoRR abs/2106.09305, arXiv:2106.09305, URL https://arxiv.org/abs/2106.09305.
[50] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, (no. 12) 2021, pp. 11106–11115.
[51] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting? in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, (no. 9) 2023, pp. 11121–11128.
[52] R. Ravichandiran, H. Bannazadeh, A. Leon-Garcia, Anomaly detection using resource behaviour analysis for autoscaling systems, in: 2018 4th IEEE Conference on Network Softwarization and Workshops, NetSoft, IEEE, 2018, pp. 192–196.
[53] A. Mehta, J. Dürango, J. Tordsson, E. Elmroth, Online spike detection in cloud workloads, in: 2015 IEEE International Conference on Cloud Engineering, 2015, pp. 446–451, http://dx.doi.org/10.1109/IC2E.2015.50.
[54] W. Sus, P. Nawrocki, Signature-based adaptive cloud resource usage prediction using machine learning and anomaly detection, J. Grid Comput. 22 (2) (2024) 46.
[55] P. Nawrocki, W. Sus, Anomaly detection in the context of long-term cloud resource usage planning, Knowl. Inf. Syst. 64 (10) (2022) 2689–2711.
[56] G. Zhao, S. Hassan, Y. Zou, D. Truong, T. Corbin, Predicting performance anomalies in software systems at run-time, ACM Trans. Softw. Eng. Methodol. (TOSEM) 30 (3) (2021) 1–33.
[57] A. Patil, R. Patil, Proactive and dynamic load balancing model for workload spike detection in cloud, Measurement: Sensors 27 (2023) 100799.
[58] R.N. Calheiros, E. Masoumi, R. Ranjan, R. Buyya, Workload prediction using ARIMA model and its impact on cloud applications' QoS, IEEE Trans. Cloud Comput. 3 (4) (2015) 449–458, http://dx.doi.org/10.1109/TCC.2014.2350475.
[59] Y. Zhang, W. Hua, Z. Zhou, G.E. Suh, C. Delimitrou, Sinan: ML-based and qos-aware resource management for cloud microservices, in: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, in: ASPLOS 2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 167–181, http://dx.doi.org/10.1145/3445814.3446693.
[60] S.G. Dastidar, K. Dutta, N. Das, M. Kundu, M. Nasipuri, Exploring knowledge distillation of a deep neural network for multi-script identification, in: International Conference on Computational Intelligence in Communications and Business Analytics, Springer, 2021, pp. 150–162.
[61] G.E. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, 2015, ArXiv abs/1503.02531, arXiv:1503.02531, URL https://api.semanticscholar.org/CorpusID:7200347.
[62] J. Wilkes, Google cluster traces data 2019, 2019. https://github.com/google/cluster-data/blob/master/ClusterData2019.md.
[63] Alibaba Group. Alibaba Cluster Trace Program, Alibaba cluster traces data 2018, 2018. https://github.com/alibaba/clusterdata/blob/master/cluster-trace-v2018/trace_2018.md.
[64] M. Barandas, D. Folgado, L. Fernandes, S. Santos, M. Abreu, P. Bota, H. Liu, T. Schultz, H. Gamboa, TSFEL: Time series feature extraction library, SoftwareX 11 (2020) 100456, http://dx.doi.org/10.1016/j.softx.2020.100456, URL https://www.sciencedirect.com/science/article/pii/S2352711020300017.
[65] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, J. Mach. Learn. Res. 12 (2011) 2825–2830.
[66] T. Head, M. Kumar, H. Nahrstaedt, G. Louppe, I. Shcherbatyi, Scikit-optimize/scikit-optimize, 2021, http://dx.doi.org/10.5281/zenodo.5565057, Zenodo.
[67] T. Rojat, R. Puget, D. Filliat, J.D. Ser, R. Gelin, N. Díaz-Rodríguez, Explainable Artificial Intelligence (XAI) on TimeSeries data: A survey, 2021, arXiv:2104.00950.
[68] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, Adv. Neural Inf. Process. Syst. 30 (2017).
[69] A. Shrikumar, P. Greenside, A. Kundaje, Learning important features through propagating activation differences, in: International Conference on Machine Learning, PMlR, 2017, pp. 3145–3153.
[70] A. Shrikumar, K. Tian, A. Shcherbina, Z. Avsec, A. Banerjee, M. Sharmin, S. Nair, A. Kundaje, TF-MoDISco v0.4.4.2-alpha: Technical note, 2018, CoRR abs/1811.00416, arXiv:1811.00416, URL http://arxiv.org/abs/1811.00416.



Ferran Agulló: received his degree in Informatics in 2020 and the M.Sc. in Artificial Intelligence in 2022. Currently, he is a Ph.D. student from the Polytechnic University of Catalonia affiliated with the Barcelona Supercomputing Center Data-Centric Computing group. He is mainly focused on two research topics, synthetic data generation for healthcare and the optimization of resource usage in high performance computing environments. He has worked on INCISIVE, CALLISTO and CLOUDSKIN European projects, and he is currently working as a researcher in the SECURED and CLOUDSTARS European projects.



Alberto Gutiérrez Torre: received his Engineering degree in Informatics (2014), M. Sc. in Data Science (2017) and Ph.D. in Computer Architectures (2022) at BarcelonaTech-UPC. He currently works in the Barcelona Supercomputing Center Data-Centric Computing group, where he works in IoT and Healthcare data analysis, from data gathering to the final analysis, and in particular his current focus is on Federated Learning and synthetic data generation for healthcare. He has worked on INCISIVE and CALLISTO European projects, and he is currently working as local Principal Investigator in the SECURED European project.



Jordi Torres: is a professor at the Technical University of Catalonia and he has a distinguished academic career encompassing teaching, management, technology transfer, and research. He has authored and coauthored over 200 research publications and books in High Performance Computing (HPC) and Artificial Intelligence (AI). He joined the Barcelona Supercomputing Center as part of the founding team and later became a research manager in the Computer Science Department. Presently, he serves as a senior researcher and Expert Advisor in HPC and AI.



Josep Lluís Berral : received his Engineering degree in Informatics (2007), M.Sc. in Computer Architecture (2008), and Ph.D. in Computer Science (2013) at BarcelonaTech-UPC. He works in high performance data-analytics and machine learning on Cloud at the Department of Computer Architecture and the Barcelona Supercomputing Center. He manages the group Computing Resources Orchestration and Management for AI. He collaborates in European projects and private agreements with Intel and IBM, and previously with Databricks, Microsoft and Cisco. He was awarded with a Juan de la Cierva research fellowship from the Spanish Ministry of Economy in 2016. He is an IEEE and ACM member.